

EXERCÍCIOS — BANCO DE DADOS

André Luiz Sales Fecury Filho

Ciência da Computação — 4º período | Campus Manaus

RA: R8330H4 | Turma: CC4P34

Prof. MSc. Mateus de Paula

Questão 1 — Banco de dados x SGBD

Resposta:

A loja deve criar um **banco de dados**, não um SGBD. O SGBD (Sistema Gerenciador de Banco de Dados, como o PostgreSQL) já é o software responsável por gerenciar o armazenamento e o acesso aos dados; a loja apenas precisa instalar um SGBD existente e, dentro dele, criar seu próprio banco de dados com as tabelas de vendas e produtos.

Questão 2 — Etapas do Projeto de Banco de Dados

Resposta:

1. **Levantamento e análise de requisitos** — entender as necessidades do negócio.
2. **Projeto conceitual** — modelagem independente de SGBD (ex.: diagrama entidade-relacionamento).
3. **Projeto lógico** — conversão do modelo conceitual para o modelo relacional (tabelas, chaves, relacionamentos).
4. **Projeto físico** — definição de índices, tipos de dados, armazenamento e otimizações específicas do SGBD escolhido.
5. **Implementação e manutenção** — criação efetiva do banco e ajustes contínuos.

Questão 3 — Propriedades da chave primária

Resposta:

A chave primária deve ser **única** (não pode haver valores repetidos), **não nula** (todo registro precisa ter valor), **imutável** (não deve mudar ao longo do tempo) e **mínima/irreduzível** (quando composta, todos os atributos são necessários para garantir a unicidade).

Questão 4 — Objetivo das chaves estrangeiras

Resposta:

Garantir a **integridade referencial** entre tabelas, assegurando que um valor referenciado em uma tabela filha exista de fato na tabela pai, evitando dados órfãos ou inconsistentes.

Questão 5 — Modelo lógico relacional (Aluno N:N Curso)

Resposta:

O relacionamento (0,n) para (0,n) entre Aluno e Curso indica cardinalidade **muitos-para-muitos** , o que exige a criação de uma tabela associativa cuja chave primária é composta pelas chaves estrangeiras de Aluno e Curso.

```
CREATE TABLE aluno (  
  id_aluno SERIAL PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL  
);  
  
CREATE TABLE curso (  
  id_curso SERIAL PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL  
);  
  
CREATE TABLE relacao1 (  
  id_aluno INT NOT NULL REFERENCES aluno(id_aluno),  
  id_curso INT NOT NULL REFERENCES curso(id_curso),  
  PRIMARY KEY (id_aluno, id_curso)  
);
```

Questão 6 — Schemas no PostgreSQL

Resposta:

Schemas são **namespaces** dentro de um mesmo banco de dados que agrupam objetos (tabelas, views, funções etc.). Servem para organizar logicamente os objetos, evitar conflitos de nomes entre aplicações diferentes e controlar permissões de acesso por grupo de objetos.

Questão 7 — Licença do PostgreSQL

Resposta:

Sim. O PostgreSQL pode ser distribuído, modificado e utilizado gratuitamente, inclusive para fins comerciais, sob a **Licença PostgreSQL (PostgreSQL License)** , uma licença de código aberto permissiva, semelhante às licenças MIT/BSD.

Questão 9 — Empresas que utilizam PostgreSQL

Resposta:

Instagram, Spotify, Apple, Netflix, Reddit, Skype e Twitch são exemplos de empresas que utilizam ou já utilizaram PostgreSQL em parte de sua infraestrutura.

Questão 10 — Banco de dados de vendedores

Resposta:

```
CREATE TABLE vendedor (  
  id          SERIAL PRIMARY KEY,  
  nome        VARCHAR(100) NOT NULL,  
  data_nascimento DATE,  
  salario     NUMERIC(10,2)  
);  
  
CREATE TABLE venda (  
  id          SERIAL PRIMARY KEY,  
  id_vendedor INT NOT NULL REFERENCES vendedor(id),  
  produto     VARCHAR(100) NOT NULL,  
  valor       NUMERIC(10,2) NOT NULL  
);
```

a) Todos os dados dos vendedores

```
SELECT * FROM vendedor;
```

b) Vendedores organizados pelo nome

```
SELECT * FROM vendedor ORDER BY nome;
```

c) Nome dos vendedores com substring 'na'

```
SELECT nome FROM vendedor WHERE nome LIKE '%na%';
```

d) Soma das vendas realizadas

```
SELECT SUM(valor) AS total_vendas FROM venda;
```

e) Soma das vendas agrupadas por vendedor

```
SELECT id_vendedor, SUM(valor) AS total_vendas  
FROM venda  
GROUP BY id_vendedor;
```

f) Nome e data de nascimento dos vendedores com alguma venda

```
SELECT v.nome, v.data_nascimento  
FROM vendedor v  
WHERE EXISTS (SELECT 1 FROM venda ve WHERE ve.id_vendedor = v.id);
```

g) Vendedor que vendeu o produto 'martelo'

```
SELECT v.nome  
FROM vendedor v  
INNER JOIN venda ve ON v.id = ve.id_vendedor  
WHERE ve.produto = 'martelo';
```

h) Total de vendedores cadastrados

```
SELECT COUNT(*) AS total_vendedores FROM vendedor;
```

i) Aumento de R\$100 no salário de vendedores com vendas > R\$1000

```
UPDATE vendedor
SET salario = salario + 100
WHERE id IN (
    SELECT id_vendedor
    FROM venda
    GROUP BY id_vendedor
    HAVING SUM(valor) > 1000
);
```

Questão 11 — Junções (JOIN)

a) Junção interna (INNER JOIN)

```
SELECT v.nome, ve.produto, ve.valor
FROM vendedor v
INNER JOIN venda ve ON v.id = ve.id_vendedor;
```

Retorna apenas os vendedores que possuem vendas associadas, combinando as duas tabelas pela chave `id_vendedor`.

b) Junção externa à direita (RIGHT JOIN)

```
SELECT v.nome, ve.produto, ve.valor
FROM vendedor v
RIGHT JOIN venda ve ON v.id = ve.id_vendedor;
```

Retorna todas as vendas, mesmo que o vendedor correspondente não exista na tabela vendedor (nome viria NULL).

c) Junção externa à esquerda (LEFT JOIN)

```
SELECT v.nome, ve.produto, ve.valor
FROM vendedor v
LEFT JOIN venda ve ON v.id = ve.id_vendedor;
```

Retorna todos os vendedores, mesmo os que não realizaram nenhuma venda (produto e valor viriam NULL).

Questão 12 — Palavra-chave para linhas únicas

Resposta: C) DISTINCT

O DISTINCT elimina linhas duplicadas do resultado de um SELECT. As demais opções têm finalidades diferentes: ALL mantém as duplicatas, enquanto ROLLUP e CUBE são usados em agregações multidimensionais.

Questão 13 — Comando que não pertence à DML

Resposta: A) ALTER TABLE

ALTER TABLE pertence à DDL (define/altera a estrutura das tabelas), enquanto INSERT, DELETE, UPDATE e SELECT manipulam ou consultam dados, fazendo parte da DML/DQL.

Questão 14 — Médico mais alto (subconsulta com IN)

Resposta:

```
CREATE TABLE medico (  
  id      SERIAL PRIMARY KEY,  
  nome    VARCHAR(50) NOT NULL,  
  altura  NUMERIC(3,2) NOT NULL  
);  
  
INSERT INTO medico (id, nome, altura) VALUES  
(1, 'M1', 1.70),  
(2, 'M2', 1.80),  
(3, 'M3', 1.90),  
(4, 'M4', 1.68);  
  
SELECT nome  
FROM medico  
WHERE altura IN (SELECT MAX(altura) FROM medico);
```

Resultado esperado:

M3

Questão 15 — Médico mais baixo após inserir M5 (subconsulta com IN)

Resposta:

```
INSERT INTO medico (id, nome, altura) VALUES (5, 'M5', 1.68);  
  
SELECT nome  
FROM medico  
WHERE altura IN (SELECT MIN(altura) FROM medico);
```

Resultado esperado:

M4
M5

M4 e M5 empatam com a menor altura (1,68 m).

Questão 16 — Médicos com consultas em dezembro/2016 (subconsulta com IN)

Resposta:

```
CREATE TABLE paciente (  
    id SERIAL PRIMARY KEY,  
    nome VARCHAR(50) NOT NULL  
);  
  
INSERT INTO paciente (id, nome) VALUES  
(1, 'P1'), (2, 'P2'), (3, 'P3');  
  
CREATE TABLE consulta (  
    id SERIAL PRIMARY KEY,  
    idPaciente INT NOT NULL REFERENCES paciente(id),  
    idMedico INT NOT NULL REFERENCES medico(id),  
    data DATE NOT NULL  
);  
  
INSERT INTO consulta (id, idPaciente, idMedico, data) VALUES  
(1, 1, 1, '2016-10-10'),  
(2, 2, 1, '2016-12-05'),  
(3, 3, 2, '2017-02-03'),  
(4, 1, 3, '2016-12-15');  
  
SELECT nome  
FROM medico  
WHERE id IN (  
    SELECT idMedico  
    FROM consulta  
    WHERE data BETWEEN '2016-12-01' AND '2016-12-31'  
);
```

Resultado esperado:

M1
M3

Questão 17 — Pacientes com idade superior a 21 anos e consulta marcada (subconsulta com IN)

Resposta:

```
ALTER TABLE paciente ADD COLUMN idade INT;  
  
UPDATE paciente SET idade = 10 WHERE id = 1;  
UPDATE paciente SET idade = 22 WHERE id = 2;  
UPDATE paciente SET idade = 38 WHERE id = 3;  
  
SELECT nome  
FROM paciente  
WHERE idade > 21  
    AND id IN (SELECT idPaciente FROM consulta);
```

Resultado esperado:

P2
P3

Questão 18 — Pacientes exceto o mais jovem (subconsulta com ANY)

Resposta:

```
SELECT nome
FROM paciente
WHERE idade > ANY (SELECT idade FROM paciente);
```

O predicado > ANY retorna verdadeiro para qualquer linha cuja idade seja maior que **pelo menos uma** idade do conjunto; isso exclui exatamente o registro com a menor idade, que não é maior que nenhum outro.

Resultado esperado:

```
P2
P3
```

Questão 19 — Médicos com salário superior a todos os pediatras (subconsulta com ALL)

Resposta:

```
ALTER TABLE medico ADD COLUMN salario NUMERIC(10,2);
ALTER TABLE medico ADD COLUMN idEspecialidade INT;

CREATE TABLE especialidade (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(50) NOT NULL
);

INSERT INTO especialidade (id, nome) VALUES
(1, 'Cardiologia'),
(2, 'Ortopedia'),
(3, 'Pediatria');

ALTER TABLE medico
    ADD CONSTRAINT fk_medico_especialidade
    FOREIGN KEY (idEspecialidade) REFERENCES especialidade(id);

UPDATE medico SET salario = 1000, idEspecialidade = 3 WHERE id = 1;
UPDATE medico SET salario = 5000, idEspecialidade = 2 WHERE id = 2;
UPDATE medico SET salario = 10000, idEspecialidade = 1 WHERE id = 3;
UPDATE medico SET salario = 12000, idEspecialidade = 3 WHERE id = 4;
UPDATE medico SET salario = 15000, idEspecialidade = 1 WHERE id = 5;

SELECT nome
FROM medico
WHERE salario > ALL (
    SELECT salario
    FROM medico
    WHERE idEspecialidade = 3
);
```

Os médicos pediatras (M1 e M4) recebem, respectivamente, 1000 e 12000. O predicado > ALL exige salário maior que o maior desses valores (12000).

Resultado esperado:

```
M5
```

Questão 20 — Pacientes com ao menos uma consulta realizada

```
INSERT INTO paciente (id, nome, idade) VALUES (4, 'P4', 44);
```

a) Subconsulta utilizando EXISTS

```
SELECT nome
FROM paciente p
WHERE EXISTS (
    SELECT 1 FROM consulta c WHERE c.idPaciente = p.id
);
```

b) Mesma subconsulta utilizando IN

```
SELECT nome
FROM paciente
WHERE id IN (SELECT idPaciente FROM consulta);
```

Resultado esperado:

```
P1
P2
P3
```

P4 não possui nenhuma consulta registrada, por isso não aparece no resultado.

Questão 21 — Estados sem fornecedores cadastrados

Resposta: C)

```
SELECT E.nome_estado
FROM Estado AS E
WHERE E.UF NOT IN (
    SELECT F.UF
    FROM Fornecedor AS F);
```

É a única alternativa correta: compara **UF com UF** (não nome com UF) usando **NOT IN**, retornando os estados cuja UF não aparece na tabela Fornecedor. As alternativas B e D têm erro de sintaxe (**FROM ... , FROM ...**); a A compara colunas incompatíveis (nome_estado com UF); a E usa **IN**, retornando o oposto do solicitado.

Questão 22 — DDL, DML, DQL, DTL e DCL

Resposta:

Sigla	Significado	Finalidade	Exemplos de comandos
DDL	Data Definition Language	Define e altera a estrutura dos objetos do banco (tabelas, schemas, índices)	CREATE, ALTER, DROP, TRUNCATE
DML	Data Manipulation Language	Manipula os dados armazenados nas tabelas	INSERT, UPDATE, DELETE
DQL	Data Query Language	Consulta os dados armazenados	SELECT
DTL	Data Transaction Language	Controla transações, garantindo atomicidade das operações	BEGIN, COMMIT, ROLLBACK, SAVEPOINT
DCL	Data Control Language	Controla permissões e acesso aos objetos do banco	GRANT, REVOKE

Exemplos compatíveis com PostgreSQL

```
-- DDL: cria e altera estrutura
CREATE TABLE especialidade (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(50) NOT NULL
);
ALTER TABLE medico ADD COLUMN salario NUMERIC(10,2);
DROP TABLE IF EXISTS tabela_temporaria;

-- DML: manipula dados
INSERT INTO paciente (id, nome, idade) VALUES (4, 'P4', 44);
UPDATE medico SET salario = salario + 100 WHERE id = 1;
DELETE FROM consulta WHERE id = 4;

-- DQL: consulta dados
SELECT nome, salario FROM medico WHERE salario > 10000;

-- DTL: controla transações
BEGIN;
UPDATE medico SET salario = salario + 500 WHERE id = 2;
COMMIT;
-- Em caso de erro: ROLLBACK;

-- DCL: controla permissões
GRANT SELECT, INSERT ON medico TO usuario_leitura;
REVOKE INSERT ON medico FROM usuario_leitura;
```